

5.3 - Fundamentos de CI/CD: integração, entrega e implantação contínuas

Três disciplinas diferentes que muitos times confundem em uma só

O ritual de sexta à noite que ninguém queria mais

- **Oito desenvolvedores, um deploy mensal, todo deploy via SSH manual em produção**
 - Sexta à noite virou sinônimo de incidente e plantão coordenado no Zoom

LUNALOG

Nos primeiros doze meses da LunaLog, deploy era um evento, não um processo. Toda última sexta-feira do mês, oito desenvolvedores entravam no mesmo Zoom às 19h, alguém ligava SSH no servidor de produção, rodava git pull e rezava. Cada semana havia conflito de merge acumulado, cada deploy reabria bugs antigos. Quando o terceiro deploy consecutivo gerou incidente em produção, Roberto cancelou o quarto deploy e disse: a partir de hoje, ninguém faz deploy manual nunca mais. Foi naquela noite que CI/CD deixou de ser teoria de blog e virou prioridade arquitetural.

Continuous Integration, o nível básico

Integrar código frequentemente com feedback rápido

- **Todo commit dispara build automatizado e bateria de testes**
 - Feedback de qualidade em minutos, não em horas ou dias
- **Branches curtas que se integram à main várias vezes por dia**
 - Quanto mais tempo uma branch fica isolada, mais cara é a integração futura
- **O pipeline de CI é o primeiro contrato de qualidade do time**
 - Build quebrado é prioridade um, todo o time para até consertar
- **CI não exige ambiente de produção, apenas disciplina de integração**

Continuous Delivery, sempre pronto para produção

Software em estado implantável a qualquer momento

- **Cada mudança que passa o pipeline gera artefato pronto para deploy**
 - Imagem Docker taguada, binário versionado, pacote publicado em registry
- **O deploy em produção continua sendo decisão humana**
 - Botão acionado por gerente de produto, por engenheiro de plantão ou por agenda
- **O sistema está sempre tecnicamente pronto, o momento é decisão de negócio**
 - Útil quando há janelas regulatórias, ciclos de marketing ou aprovações formais
- **Pré-requisito é CI maduro com cobertura de teste confiável**

Continuous Deployment, automático até produção

Cada commit aprovado chega ao usuário sem intervenção humana

- **Cada commit que passa todos os gates vai automaticamente para produção**
 - Não há aprovação manual, não há janela de deploy, não há cerimônia
- **Exige observabilidade madura para detectar problemas em minutos**
 - SLOs definidos, alertas automatizados, rollback automatizado
- **Feature flags se tornam obrigatórias para desacoplar deploy de release**
 - Funcionalidade pode estar no código sem estar ativa para os usuários
- **Cultura de pequenas mudanças, não de grandes releases**
 - Dez deploys pequenos por dia são mais seguros que um grande por semana

Delivery contra Deployment, a diferença crítica

Continuous Delivery

- Pipeline gera artefato implantável a cada mudança
- Decisão de deploy continua sendo humana
- Adequado para domínios com janelas regulatórias
- Botão acionado por papel definido na organização
- Exige CI maduro como pré-requisito

Continuous Deployment

- Pipeline empurra cada mudança aprovada para produção
- Decisão de deploy é totalmente automática
- Adequado para produtos digitais com mudanças frequentes
- Sem botão, sem cerimônia, sem janela
- Exige observabilidade e rollback automatizados

Cada nível tem pré-requisitos não negociáveis

- **Pular níveis sem cobrir os pré-requisitos cria dívida operacional severa**

CI exige disciplina de integração e testes confiáveis. Continuous Delivery exige CI maduro e ambiente de staging fiel à produção. Continuous Deployment exige observabilidade, alertas, rollback automatizado e feature flags. Times que adotam Continuous Deployment sem essas bases não estão automatizando, estão acelerando o desastre.

- Avalie maturidade antes de declarar nível de adoção, não o contrário



Os fundamentos de CI/CD estão estabelecidos, mas o que separa um pipeline maduro de um que apenas automatiza o caos? Cada stage de um pipeline tem responsabilidade definida, cada quality gate tem propósito claro, e cada artefato gerado precisa ser auditável. Sem essa anatomia, automação vira velocidade sem direção.

Anatomia de um pipeline maduro, stages, gates e artefatos